

XPTO · DEVOPS RUNBOOK

Pipeline CI/CD

Rock in Rio — Backend

Documentação operacional completa do processo de build, teste, segurança e deploy da API **Achados & Perdidos** (Spring Boot + Docker + GitLab CI).

Aplicação: api-achados-perdidos · Spring Boot 4.0.2 / Java 25

Orquestração CI/CD: GitLab CI/CD · runner com tag `deploy`

Ambientes: `develop` → Development · `main` → Production

Público-alvo: Equipe DevOps / SRE responsável pela execução e manutenção do pipeline

Emissão: 23/07/2026 · Versão 1.0

Sumário

- 01 Sumário executivo e visão geral
- 02 Stack tecnológica e topologia de deploy
- 03 Pré-requisitos de infraestrutura (runner e VPS)
- 04 Variáveis de ambiente e gestão de segredos
- 05 Anatomia do pipeline — `.gitlab-ci.yml`
- 06 Stage **build** — imagem e extração do fat-jar
- 07 Stage **test** — MySQL efêmero e gate real
- 08 Stage **security** — SCA, SAST e secret scanning
- 09 Stage **deploy** — publicação em dev e produção
- 10 Dockerfile multi-stage e runtime (compose)
- 11 Nginx, Cloudflare e exposição da API
- 12 Banco de dados e migrations SQL
- 13 Procedimento de bootstrap (primeiro deploy)
- 14 Deploy contínuo, rollback e limpeza
- 15 Observabilidade e troubleshooting
- 16 Checklist operacional e hardening

1. Sumário executivo e visão geral

Este runbook descreve, de ponta a ponta, como o backend da aplicação **Achados & Perdidos** (Rock in Rio) é construído, testado, auditado e implantado automaticamente pelo GitLab CI/CD. O objetivo é permitir que qualquer pessoa da equipe DevOps execute, diagnostique e evolua o pipeline sem depender de conhecimento tácito.

4

estágios sequenciais: **build** → **test** → **security** → **deploy**

2

ambientes por branch: **develop** (dev) e **main** (produção)

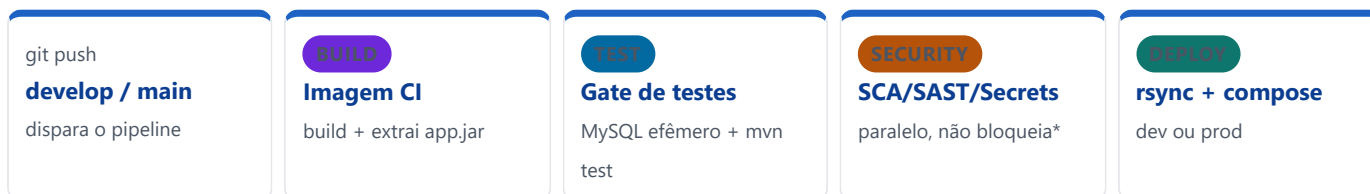
3

scanners de segurança: Trivy (SCA), Semgrep (SAST), Gitleaks (segredos)

1

Dockerfile multi-stage (**base** → **test** → **build** → **runtime**)

1.1 Fluxo macro do pipeline



* Os jobs de segurança rodam com `allow_failure: true` — são **visíveis** mas não bloqueiam o deploy até que a equipe faça a primeira triagem de findings (ver §8).

1.2 Princípios de design adotados

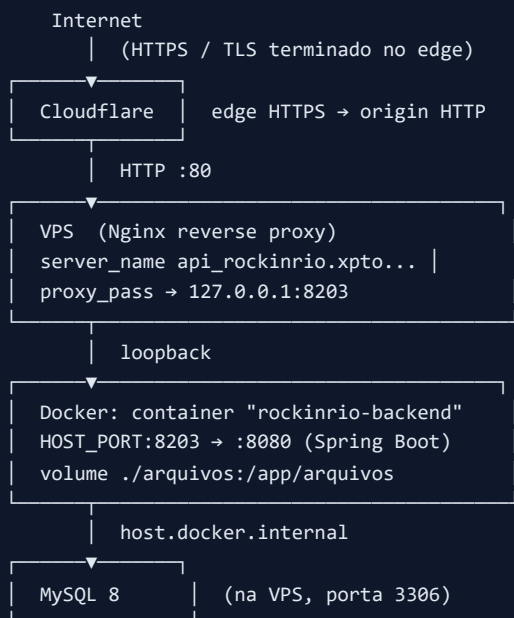
- **Gate de teste real:** os testes de integração dependem de um MySQL de verdade. Sem banco, a suíte é *pulada* (`assumeTrue`) e o gate não validaria nada — por isso o CI sobe um MySQL efêmero antes de testar.
- **Build reproduzível e offline no SCA:** o Trivy escaneia o **fat-jar já construído**, sem re-resolver o Maven Central (evita rate-limit HTTP 429).
- **Deploy imutável por commit:** cada imagem é etiquetada com `ci-${CI_COMMIT_SHORT_SHA}`, permitindo rastreabilidade e rollback.
- **Segurança por camadas (OWASP):** SCA (A06), SAST (A05/A08), secret scanning, além de rate-limiting, JWT com fail-fast e CORS restritivo na aplicação.
- **Isolamento por ambiente:** `resource_group` serializa deploys concorrentes por ambiente, evitando corrida entre pipelines.

2. Stack tecnológica e topologia de deploy

2.1 Stack da aplicação

Camada	Tecnologia	Observações
Linguagem / Runtime	Java 25	Compilado e executado sobre Eclipse Temurin 25
Framework	Spring Boot 4.0.2	Web MVC, Data JPA, Security, Validation, Mail
Segurança	Spring Security + JWT (jjwt 0.12.6)	IDs opacos assinados, Bucket4j para rate-limit
Persistência	MySQL 8 · Hibernate	Banco <code>achados_perdidos</code> ; <code>ddl-auto=validate</code> em produção
Armazenamento	Disco local ou AWS S3	SDK AWS 2.31.78; provider configurável em <code>/configuracoes</code>
Documentação	springdoc-openapi 3.0.2 (Swagger UI)	Desligável via <code>SPRINGDOC_ENABLED=false</code> em prod
Build	Maven 3.9	<code>finalName=app</code> → artefato <code>target/app.jar</code>

2.2 Topologia de produção



Domínio com underscore

`api_rockinrio.xpto.com.br` contém `_`, que o Tomcat **rejeita** no header `Host` (HTTP 400). O Nginx envia ao upstream um `Host` válido com hífen (`api-rockinrio...`) e preserva o domínio real em `X-Forwarded-Host`. `X-Forwarded-Proto: https` informa à aplicação que o TLS foi terminado no Cloudflare.

2.3 Diretórios-chave no repositório

Caminho	Papel no pipeline
<code>.gitlab-ci.yml</code>	Definição completa do pipeline (stages, jobs, rules)
<code>Dockerfile</code>	Build multi-stage: base → test → build → runtime
<code>docker-compose.yml</code>	Runtime do container no servidor de deploy
<code>.dockerignore</code> / <code>.gitleaks.toml</code>	Contexto de build enxuto / allowlist de secret scanning
<code>scripts/*.sql</code>	Migrations versionadas (018 ... 054) aplicadas ao MySQL
<code>scripts/ci-docker-cleanup.sh</code>	Limpeza pós-deploy (imagens/cache antigos)
<code>deploy/nginx/*.conf</code>	Vhost do reverse proxy (referência de configuração)
<code>.env</code> (no servidor)	Segredos e configuração de runtime — NÃO versionado

3. Pré-requisitos de infraestrutura

3.1 GitLab Runner

Todos os jobs declaram `tags: [deploy]`. É obrigatório existir ao menos um runner registrado com essa tag, com acesso ao Docker daemon.

- **Executor:** Shell ou Docker com socket montado — o pipeline invoca `docker build`, `docker run`, `docker compose` diretamente.
- **Imagem default dos jobs:** `docker:24-cli` com `pull_policy: if-not-present`.
- **Docker Engine** + plugin **Compose v2** (o pipeline usa `docker compose`, não `docker-compose`).
- **Localização:** o mesmo runner que testa também faz o deploy — ele precisa enxergar o diretório `/opt/rockinrio-backend` na VPS e o MySQL de produção.
- **Rede:** acesso de saída à internet para baixar imagens (Trivy DB, Semgrep, Gitleaks, MySQL, Temurin).

Por que o runner é local à VPS

O template de deploy faz `rsync` do repositório para `/opt/rockinrio-backend` e sobe o compose **na própria máquina do runner**. Ou seja, o runner com a tag `deploy` deve rodar **no servidor de destino** (ou ter o diretório/daemon compartilhados). O stage `test` também usa `--network host`, o que pressupõe runner Linux com host networking.

3.2 Servidor de deploy (VPS)

Requisito	Detalhe
Sistema	Linux com Docker Engine 24+ e Compose v2
Diretório de deploy	<code>/opt/rockinrio-backend</code> (variável <code>DEPLOY_DIR</code>)
Arquivo de segredos	<code>/opt/rockinrio-backend/.env</code> — criado manualmente antes do 1º deploy
MySQL	Instância acessível via <code>host.docker.internal</code> (mapeado por <code>host-gateway</code>) na porta 3306
Nginx	Reverse proxy para <code>127.0.0.1:8203</code> (produção) — ver §11
Portas	3306 ocupada pelo banco; o container publica em <code>HOST_PORT</code> (dev 8080, prod 8203)
Ferramentas	<code>rsync</code> (instalado automaticamente pelo <code>before_script</code> se ausente)

4. Variáveis de ambiente e gestão de segredos

4.1 Variáveis definidas no pipeline

Variável	Valor	Papel
DEPLOY_DIR	/opt/rockinrio-backend	Destino do rsync e raiz do compose no servidor
COMPOSE_PROJECT_NAME	rockinrio-backend	Namespace do projeto Docker Compose
IMAGE_CI	rockinrio-backend:ci-\${CI_COMMIT_SHORT_SHA}	Tag da imagem por commit (build/cleanup)
IT_MYSQL_PORT	13306	Porta de host do MySQL efêmero de teste (3306 já usada pela VPS)

4.2 O arquivo .env no servidor (runtime)

O `.env` vive apenas em `/opt/rockinrio-backend/.env`, **fora do git e preservado pelo rsync** (`--exclude .env`). É a fonte de todos os segredos de runtime consumidos pelo `docker-compose.yml` via `env_file`. Base em `.env.example`:

Grupo	Chaves	Notas críticas
Banco	DB_HOST , DB_PORT , DB_NAME , DB_URL , DB_USERNAME , DB_PASSWORD	No compose, <code>DB_HOST</code> é sobrescrito para <code>host.docker.internal</code>
Segredos (obrigatórios)	JWT_SECRET , RESOURCE_ID_SECRET	Mín. 32 chars. A app NÃO sobe sem eles (fail-fast). Gerar com <code>openssl rand -base64 48</code>
JWT	JWT_EXPIRATION_MS , JWT_REFRESH_EXPIRATION_MS	Access 24h / refresh 30d por padrão
Swagger	SPRINGDOC_ENABLED	false em produção
Rate limit	LOGIN_RATE_LIMIT_* , PUBLIC_RATE_LIMIT_*	Anti força-bruta (login) e anti-abuso (portal público)
CORS	CORS_ALLOWED_ORIGINS , CORS_ALLOWED_ORIGIN_PATTERNS , CORS_ALLOW_CREDENTIALS	Em prod: domínios REAIS e esvaziar patterns de localhost
Arquivos	APP_ARQUIVOS_DIR , MULTIPART_MAX_REQUEST_SIZE	Dir montado como volume <code>./arquivos</code>
S3 (opcional)	APP_S3_* , AWS_ACCESS_KEY_ID , AWS_SECRET_ACCESS_KEY	Credenciais via env ou IAM role; nunca no banco
Runtime	HOST_PORT	Porta publicada pelo container (dev 8080 · prod 8203)

Fail-fast de segredos

Em `application.properties`, `app.jwt.secret` e `app.resource-id.secret` não têm valor default. Se o `.env` não os fornecer, a aplicação **não inicia** — o container entrará em loop de restart. Sempre validar o `.env` antes de subir.

4.3 Como o segredo NÃO vaza no CI

- O `.env` é excluído do rsync e do contexto de build (`.dockerignore`) — nunca entra na imagem.
- Gitleaks** escaneia o histórico e a árvore em busca de segredos commitados; `.gitleaks.toml` mantém uma allowlist só para placeholders de exemplo.
- Credenciais de produção devem viver como **CI/CD variables mascaradas/protegidas** no GitLab (se injetadas via pipeline) ou diretamente no `.env` do servidor.

5. Anatomia do pipeline — .gitlab-ci.yml

5.1 Estágios e ordem de execução

```
stages:
- build      # 1. Constrói a imagem e extrai o fat-jar
- test       # 2. Gate de testes com MySQL efêmero
- security   # 3. SCA + SAST + secret scanning (paralelo)
- deploy     # 4. rsync + docker compose up (dev/prod)
```

5.2 Regras de branch (quando o pipeline roda)

Uma âncora YAML reutilizável restringe a execução às branches `main` e `develop`. Qualquer outra branch (feature, hotfix) **não dispara** build/test/security.

```
.branch_main_develop: &branch_main_develop
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_COMMIT_BRANCH == "develop"
```

5.3 Grafo de dependências (`needs`)

Job	Stage	needs	Roda em
build	build	—	main, develop
test	test	build	main, develop
sca_dependencias	security	build	main, develop
sast_semgrep	security	build	main, develop
secret_detection	security	build	main, develop
deploy_development	deploy	test	develop
deploy_production	deploy	test	main

Observação importante

O deploy depende **apenas de** `test` (não dos jobs de segurança). Como os jobs de segurança são `allow_failure`, o deploy prossegue mesmo com findings — até a equipe promovê-los a gate bloqueante (§8.4).

6. Stage **BUILD** — imagem e extração do fat-jar

```
build:
  stage: build
  tags: [deploy]
  <<: *branch_main_develop
  script:
    - docker build -t "${IMAGE_CI}" .
    # Extraí o fat-jar para o SCA (Trivy) escanear offline
    - docker rm -f "ci-extract-${CI_JOB_ID}" >/dev/null 2>&1 || true
    - docker create --name "ci-extract-${CI_JOB_ID}" "${IMAGE_CI}"
    - docker cp "ci-extract-${CI_JOB_ID}:/app/app.jar" ./app.jar
    - docker rm -f "ci-extract-${CI_JOB_ID}" >/dev/null 2>&1 || true
  artifacts:
    paths: [ app.jar ]
    expire_in: 1 hour
```

6.1 O que acontece, passo a passo

1. **docker build** executa o Dockerfile até o estágio final (runtime), produzindo a imagem `rockinnrio-backend:ci-<sha>`. Nesse processo o estágio `build` roda `mvn -DskipTests package`, gerando `target/app.jar`.
2. Um container temporário é **criado** (não iniciado) a partir da imagem só para copiar o artefato de dentro dele.
3. `docker cp` extrai `/app/app.jar` para o workspace do CI.
4. O container temporário é removido; o `app.jar` vira **artefato** (validade 1h) consumido pelo job de SCA.

Por que extrair o jar?

O Trivy escaneia o fat-jar **offline** — todas as dependências já estão embarcadas nele. Isso evita que o scanner re-resolva o Maven Central e leve um [HTTP 429](#) (rate-limit), tornando o SCA determinístico e rápido.

6.2 Falhas comuns neste stage

Sintoma	Causa provável	Ação
Build lento / baixa muitas deps	Cache de layer perdido	Normal no primeiro build; layers <code>go-offline</code> são cacheadas depois
<code>cannot connect to Docker daemon</code>	Runner sem acesso ao socket	Verificar executor/registo do runner (\$3.1)
Falha no <code>docker cp</code>	Mudou o <code>finalName</code> ou o caminho do jar	Garantir <code>finalName=app</code> no pom e <code>/app/app.jar</code> no runtime stage

7. Stage **TEST** — MySQL efêmero e gate real

```
test:
  stage: test
  tags: [deploy]
  needs: [build]
  variables:
    IT_MYSQL_PORT: "13306" # 3306 já usada pelo banco da VPS
  script:
    - docker run -d --name "it-mysql-${CI_JOB_ID}"
      -e MYSQL_ALLOW_EMPTY_PASSWORD=1
      -e MYSQL_DATABASE=achados_perdidos_it
      -p ${IT_MYSQL_PORT}:3306 mysql:8.0
      --default-authentication-plugin=mysql_native_password
    - for i in $(seq 1 40); do docker exec ... mysqladmin ping ... && break; sleep 2; done
    - docker build --target test --network host
      --build-arg TEST_DB_URL="jdbc:mysql://127.0.0.1:${IT_MYSQL_PORT}/achados_perdidos_it?..."
      -t "rockinrio-backend-test:${CI_COMMIT_SHORT_SHA}" .
  after_script:
    - docker rm -f "it-mysql-${CI_JOB_ID}" >/dev/null 2>&1 || true
```

7.1 Por que um MySQL efêmero é indispensável

A base de testes (`IntegrationTestBase.java`) usa `assumeTrue(canConnect(), ...)` no `@BeforeAll`. Se não houver banco alcançável, os testes de integração são **pulados silenciosamente** — o gate ficaria verde sem validar nada. Para garantir um gate *real*, o CI:

1. Sobe um container `mysql:8.0` descartável na porta **13306** do host (a 3306 é do banco de produção da VPS).
2. Aguarda o MySQL aceitar conexões via `mysqladmin ping` (até 40 tentativas × 2s = 80s de timeout).
3. Executa `docker build --target test`, que dispara `RUN mvn -B test` dentro do estágio `test` do Dockerfile.
4. Usa `--network host` para que o `mvn test` (rodando no build) alcance o MySQL em `127.0.0.1:13306`, passado via `--build-arg TEST_DB_URL`.
5. No `after_script` (executa mesmo em falha), o MySQL efêmero é removido — sem lixo entre pipelines.

Este é o gate bloqueante

Se qualquer teste falhar, o `docker build --target test` retorna erro e o **stage test falha**, impedindo o deploy (que depende de `test`). É o principal portão de qualidade do pipeline.

7.2 Cobertura de testes executada

Suíte	Tipo	Requer MySQL?
AchadosPerdidosApplicationTests	Contexto (H2 em memória)	Não
AuthIntegrationTest	Integração — autenticação/JWT	Sim
FluxoPrincipalIntegrationTest	Integração — fluxo de negócio	Sim
PortalIntegrationTest	Integração — portal público	Sim
LocalIntegrationTest / LogsAcessoIntegrationTest	Integração	Sim
Testes de service/storage/util	Unitários	Não

O schema dos testes é criado com `ddl-auto=create` (banco `achados_perdidos_it`), independente das migrations de produção.

8. Stage **SECURITY** — SCA, SAST e secret scanning

Três jobs rodam **em paralelo**, todos com `needs: [build]` e `allow_failure: true`. São uma "avaliação contínua de risco": ficam **visíveis** (vermelhos quando há achados) mas **não bloqueiam** o deploy até a triagem inicial.

8.1 SCA — Trivy (OWASP A06: dependências vulneráveis)

```
sca_dependencias:
  image: aquasec/trivy:latest
  needs: [build]
  cache: { key: trivy-db, paths: [ .trivycache/ ] }
  script:
    - trivy fs --scanners vuln --severity HIGH,CRITICAL
      --exit-code 1 --format table app.jar
  allow_failure: true
```

Escaneia o **fat-jar** (artefato do build) buscando CVEs conhecidos nas dependências embarcadas. `--exit-code 1` deixa o job vermelho quando há HIGH/CRITICAL. O `.trivycache/` é cacheado para não rebaixar a base de vulnerabilidades toda vez.

8.2 SAST — Semgrep (A05/A08: código inseguro)

```
sast_semgrep:
  image: semgrep/semgrep:latest
  needs: [build]
  script:
    - semgrep scan --error
      --config "p/java" --config "p/secrets" --config "p/owasp-top-ten"
      --exclude-rule dockerfile.security.missing-user.missing-user .
  allow_failure: true
```

Análise estática do código-fonte com regras públicas de Java, segredos e OWASP Top 10. A regra `missing-user` (rodar container como root) é **excluída por ora**: o compose faz bind-mount de `./arquivos`, e rodar como não-root exigiria ajustar permissões do volume no servidor.

8.3 Secret scanning — Gitleaks

```
secret_detection:
  image: zricethezav/gitleaks:latest
  needs: [build]
  script:
    - gitleaks detect --source . --config .gitleaks.toml
      --redact --verbose --exit-code 1
  allow_failure: true
```

Procura chaves, tokens e senhas commitados. A allowlist (`.gitleaks.toml`) cobre apenas arquivos `.example`, placeholders e valores que a própria aplicação bloqueia em produção — reduzindo falsos-positivos.

8.4 Como promover um job a GATE bloqueante

Procedimento

Após a primeira triagem dos findings, remova `allow_failure: true` do job desejado (ex.: `sca_dependencias`). A partir daí, um HIGH/CRITICAL fará o pipeline falhar. Recomenda-se começar pelo **secret_detection** (tolerância zero) e depois SCA.

8.5 Mapa OWASP x controle

OWASP	Risco	Controle no pipeline / app
A02	Falhas criptográficas	Fail-fast de <code>JWT_SECRET</code> / <code>RESOURCE_ID_SECRET</code>
A05	Configuração insegura	Semgrep (SAST) + Swagger off em prod
A06	Componentes vulneráveis	Trivy (SCA) sobre o fat-jar
A07	Falhas de autenticação	Bucket4j rate-limit de login + secret scanning
A08	Integridade de dados/CI	SAST + imagem por commit + gate de teste

9. Stage **DEPLOY** — publicação em dev e produção

```
.deploy_template: &deploy_template
stage: deploy
before_script:
  - # instala rsync (apk/apt) se ausente
script:
  - mkdir -p "${DEPLOY_DIR}"
  - rsync -a --delete --no-owner --no-group
    --exclude .env --exclude .git --exclude target
    --exclude arquivos --exclude paineladmin --exclude portalpublico
    ./ "${DEPLOY_DIR}/"
  - cd "${DEPLOY_DIR}"
  - test -f .env || (echo "ERRO - crie .env..." && exit 1)
  - docker compose down --remove-orphans --timeout 30 || true
  - docker rm -f rockinrio-backend 2>/dev/null || true
  - docker compose up -d --build --remove-orphans --force-recreate
  - docker compose ps
  - IMAGE_CI="${IMAGE_CI}" sh scripts/ci-docker-cleanup.sh
```

9.1 Passo a passo do deploy

1. **rsync** sincroniza o repositório para `/opt/rockinrio-backend`. As exclusões protegem o `.env`, os `arquivos` (uploads dos usuários), o build local e os frontends.
2. Valida a presença do `.env` — aborta se não existir (proteção contra subir sem segredos).
3. **Para** a stack atual (`compose down`) e remove qualquer container órfão com o nome fixo.
4. **Sobe** a nova versão com `--build --force-recreate` : reconstrói a imagem a partir do código recém-sincronizado e recria o container.
5. `docker compose ps` imprime o estado final para inspeção no log do job.
6. Executa a limpeza (`ci-docker-cleanup.sh`): remove imagens CI antigas (mantém as 3 mais recentes), containers parados >24h e cache de build >168h. **Nunca** remove volumes ou imagens em uso.

9.2 Jobs concretos por ambiente

	deploy_development	deploy_production
Branch (rule)	<code>develop</code>	<code>main</code>
resource_group	rockinrio-backend-dev	rockinrio-backend-prod
Environment	development	production
URL	.../swagger-ui.html	— (Swagger desligado)
HOST_PORT típico	8080	8203

resource_group

Serializa deploys por ambiente: dois pipelines na mesma branch não executam o deploy simultaneamente, evitando corrida no `compose up` e estados inconsistentes.

Estratégia de indisponibilidade

O deploy usa `down` → `up --build` : há uma **janela de downtime** enquanto a imagem é reconstruída e o container reinicia. Para janelas curtas, planeje deploys em horários de baixo tráfego do evento. Blue-green não está implementado neste pipeline.

10. Dockerfile multi-stage e runtime

```
FROM maven:3.9-eclipse-temurin-25 AS base
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline      # camada cacheável de dependências
COPY src ./src

FROM base AS test                    # usado só no stage "test" do CI
ARG TEST_DB_URL
ENV TEST_DB_URL=${TEST_DB_URL} ...
RUN mvn -B test                      # falha aqui = gate de teste vermelho

FROM base AS build
RUN mvn -B -DskipTests package       # gera target/app.jar

FROM eclipse-temurin:25-jre-alpine  # runtime enxuto (JRE only)
WORKDIR /app
ENV SERVER_PORT=8080
COPY --from=build /app/target/app.jar app.jar
RUN mkdir -p arquivos
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

10.1 Papel de cada estágio

Estágio	Base	Quando roda	Saída
base	maven:3.9-temurin-25	Sempre (fundação)	Deps offline + código-fonte
test	base	Só no CI (<code>--target test</code>)	Resultado do <code>mvn test</code>
build	base	No build da imagem final	<code>target/app.jar</code>
runtime (final)	temurin:25-jre-alpine	Sempre	Imagem final ~JRE + app.jar

O uso de JRE Alpine no estágio final reduz drasticamente o tamanho e a superfície de ataque da imagem (sem Maven, sem JDK completo, sem código-fonte).

10.2 Runtime — `docker-compose.yml`

```
services:
  backend:
    build: .
    image: rockinrio-backend:latest
    container_name: rockinrio-backend
    restart: unless-stopped
    ports:
      - "${HOST_PORT:-8080}:8080"
    env_file: [ .env ]
    environment:
      SERVER_PORT: "8080"
      DB_HOST: host.docker.internal  # alcança o MySQL da VPS
      APP_ARQUIVOS_DIR: /app/arquivos
    extra_hosts:
      - "host.docker.internal:host-gateway"
    volumes:
      - ./arquivos:/app/arquivos    # uploads persistidos no host
```

- **restart: unless-stopped** — o container volta sozinho após reboot ou crash (mas fica em loop se o `.env` estiver incompleto — ver §4.2).

- **host.docker.internal + host-gateway** — permite que o container acesse o MySQL que roda na própria VPS.
- **volume ./arquivos** — uploads dos usuários (CLAIM, CLAIM_MENSAGEM, ITEM) sobrevivem a reddeploys; por isso o rsync **não** sincroniza essa pasta.

11. Nginx, Cloudflare e exposição da API

Referência em `deploy/nginx/api-rockinrio.xpto.com.br.conf`. Deve ser instalada em `/etc/nginx/sites-available/` com symlink em `sites-enabled/`.

```
server {
    listen 80;
    server_name api-rockinrio.xpto.com.br;
    location / {
        proxy_pass http://127.0.0.1:8203;      # HOST_PORT do container (prod)
        proxy_http_version 1.1;
        proxy_set_header Host                api-rockinrio.xpto.com.br; # hífen: Tomcat rejeita "-"
        proxy_set_header X-Forwarded-Host    api-rockinrio.xpto.com.br; # domínio real
        proxy_set_header X-Real-IP           $remote_addr;
        proxy_set_header X-Forwarded-For     $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto   https; # TLS terminado no Cloudflare
        proxy_read_timeout 60s;
        client_max_body_size 20m;
    }
}
```

11.1 Instalação / recarga

```
sudo ln -s /etc/nginx/sites-available/api-rockinrio.conf /etc/nginx/sites-enabled/
sudo nginx -t          # valida a sintaxe
sudo systemctl reload nginx
```

Consistência de portas e limites

Alinhe três valores: `HOST_PORT` no `.env`, `proxy_pass` no Nginx e a porta física livre na VPS. Além disso, `client_max_body_size` (Nginx, 20m) deve acompanhar `MULTIPART_MAX_REQUEST_SIZE` (app) — caso contrário uploads grandes tomam 413 no proxy antes de chegar à aplicação.

A aplicação confia em headers de proxy via `server.forward-headers-strategy=NATIVE` (Tomcat RemoteIpValve), refletindo o IP real do cliente sem aceitar spoofing (relevante para rate-limit e auditoria).

12. Banco de dados e migrations SQL

As migrations NÃO são aplicadas pelo pipeline

A aplicação roda com `spring.jpa.hibernate.ddl-auto=validate` em produção — o Hibernate **valida** que o schema bate com as entidades, mas **não cria nem altera** tabelas. Os scripts `scripts/0XX_*.sql` devem ser aplicados **manualmente** ao MySQL, em ordem numérica, **antes** de subir a versão que os requer. Se o schema divergir das entidades, a aplicação **não inicia** (falha de validação).

12.1 Ordem e natureza

Os scripts vão de `018_Fluxo_Processos_Documento.sql` a `054_Item_Tags.sql` (numeração incremental). Contêm DDL (tabelas, índices, FKs), seeds e correções. Devem ser aplicados **na ordem do número**, cada um uma única vez.

12.2 Procedimento recomendado de migration

1. **Backup** antes de qualquer alteração: `mysqldump achados_perdidos > backup_$(date +%F).sql`
2. Identifique o último script já aplicado (mantenha um registro/tabela de controle).
3. Aplique os novos em ordem: `mysql achados_perdidos < scripts/0XX_*.sql`
4. **Só então** faça o merge/push que dispara o deploy da nova versão da aplicação.

Sequência crítica de release

A regra de ouro: **migration primeiro, deploy depois**. Como o deploy é automático no push para `main / develop`, aplique o SQL no banco **antes** de fazer o merge. Caso contrário, o novo container falhará na validação de schema e entrará em loop de restart.

13. Procedimento de bootstrap (primeiro deploy)

Execute **uma única vez** ao preparar um ambiente novo (dev ou prod). Depois disso, os deploys são automáticos via push.

13.1 Passos

1. **Provisionar a VPS:** Docker Engine 24+, Compose v2, MySQL 8 acessível, Nginx.
2. **Registrar o GitLab Runner** com a tag `deploy`, com acesso ao Docker daemon e ao diretório de deploy.
3. **Criar o diretório:** `sudo mkdir -p /opt/rockinrio-backend`
4. **Criar o banco:** `CREATE DATABASE achados_perdidos;` e aplicar as migrations `018..054` em ordem (§12).
5. **Criar o `.env`** em `/opt/rockinrio-backend/.env` a partir do `.env.example`, com valores REAIS:

```
JWT_SECRET=$(openssl rand -base64 48)
RESOURCE_ID_SECRET=$(openssl rand -base64 48)
DB_PASSWORD=<senha real do MySQL>
SPRINGDOC_ENABLED=false           # produção
CORS_ALLOWED_ORIGINS=https://painel_rockinrio...,https://app_rockinrio...
CORS_ALLOWED_ORIGIN_PATTERNS=    # vazio em produção
HOST_PORT=8203                    # produção
```

6. **Definir a senha do admin** (BCrypt) no banco:

```
UPDATE usuario SET NM_Senha = '$2a$10$...' WHERE NM_Login = 'admin';
```

7. **Configurar o Nginx** (§11) e recarregar.
8. **Disparar o primeiro deploy:** push/merge para `develop` (dev) ou `main` (prod). O pipeline fará rsync + compose up.
9. **Validar** (§15): `docker compose ps`, logs, e uma requisição de saúde através do Nginx.

Checklist de sanidade pré-primeiro-deploy

☐ `.env` existe e tem JWT/RESOURCE secrets ≥32 chars · ☐ banco criado e migrations aplicadas · ☐ MySQL alcançável via `host.docker.internal` · ☐ porta `HOST_PORT` livre · ☐ Nginx apontando para a porta certa · ☐ runner com tag `deploy` online.

14. Deploy contínuo, rollback e limpeza

14.1 Deploy contínuo (dia a dia)

1. (Se houver mudança de schema) aplicar migrations no banco — §12.
2. `git push` para `develop` → deploy automático em **Development**.
3. Após validação em dev, merge para `main` → deploy automático em **Production**.
4. Acompanhar o job de deploy no GitLab e a saída de `docker compose ps`.

14.2 Rollback

O pipeline não tem botão de rollback nativo; use uma das estratégias:

Estratégia	Como	Quando
Revert de commit	<code>git revert <sha></code> + push → redeploy da versão anterior	Regressão de código; forma mais limpa
Re-run de pipeline antigo	No GitLab, re-executar o pipeline de um commit estável anterior	Quando o código anterior já passou pelo gate
Imagem CI anterior	<code>docker tag rockinrio-backend:ci-<sha_antigo> ...</code> e subir manualmente	Emergência; a imagem ainda existe (cleanup mantém as 3 últimas)

Rollback e banco

Se o release avançou o schema (migration nova), o rollback de código pode **falhar na validação** contra o schema novo. Migrations idealmente são retrocompatíveis (aditivas). Reverter schema exige script de *down* manual e backup — planeje antes.

14.3 Limpeza automática pós-deploy

`scripts/ci-docker-cleanup.sh` roda ao fim de todo deploy:

- Remove containers parados há mais de 24h e imagens dangling.
- Mantém as `KEEP_CI_IMAGES` (default **3**) imagens `ci-*` mais recentes; remove as demais.
- Poda o cache de build mais velho que `BUILD_CACHE_UNTIL` (default **168h**) e redes órfãs.
- **Nunca** remove volumes nem imagens em uso — os uploads e o banco estão seguros.

15. Observabilidade e troubleshooting

15.1 Comandos de diagnóstico no servidor

```
cd /opt/rockinrio-backend
docker compose ps          # estado do container
docker compose logs -f --tail=200 backend # logs em tempo real
docker inspect rockinrio-backend --format '{{.State.Health.Status}}' '{{.State.Status}}'
docker stats rockinrio-backend --no-stream # CPU/memória
curl -H "Host: api-rockinrio.xpto.com.br" http://127.0.0.1:8203/swagger-ui.html # dev
```

15.2 Matriz de sintomas

Sintoma	Causa provável	Ação
Container em loop de restart	<code>JWT_SECRET</code> / <code>RESOURCE_ID_SECRET</code> ausentes ou <32 chars	Corrigir <code>.env</code> , <code>docker compose up -d --force-recreate</code>
App sobe e cai — erro de schema	Migration não aplicada (<code>ddl-auto=validate</code>)	Aplicar SQL pendente (§12) e recriar
HTTP 400 no Nginx	Header <code>Host</code> com underscore chegando ao Tomcat	Conferir <code>proxy_set_header Host</code> com hífen
HTTP 413 em upload	<code>client_max_body_size</code> < tamanho do arquivo	Alinhar Nginx e <code>MULTIPART_MAX_REQUEST_SIZE</code>
Não conecta ao banco	<code>host.docker.internal</code> não resolve / MySQL down	Verificar <code>extra_hosts</code> e o serviço MySQL
CORS bloqueado no frontend	Origem não listada em <code>CORS_ALLOWED_ORIGINS</code>	Adicionar domínio real; recriar container
Deploy trava concorrente	<code>resource_group</code> serializando	Aguardar o pipeline anterior concluir
Stage test falha só às vezes	MySQL efêmero não subiu no timeout (80s)	Re-executar; verificar carga do runner

15.3 Onde olhar primeiro

1. **Log do job GitLab** — falha de build/test/deploy aparece aqui.
2. `docker compose logs` — falha de runtime da aplicação (schema, secret, banco).
3. `nginx -t` e **log do Nginx** — problemas de proxy/headers/TLS.
4. **MySQL** — conectividade e integridade do schema.

16. Checklist operacional e hardening

16.1 Checklist antes de cada release em produção

- ☐ Migrations necessárias aplicadas ao banco de produção (em ordem).
- ☐ Backup do banco realizado.
- ☐ `SPRINGDOC_ENABLED=false` no `.env` de produção.
- ☐ `CORS_ALLOWED_ORIGINS` só com domínios reais; patterns de localhost vazios.
- ☐ Gate de teste verde no pipeline.
- ☐ Findings de segurança triados (Trivy/Semgrep/Gitleaks).
- ☐ Janela de baixo tráfego (há downtime no `down/up`).
- ☐ Plano de rollback definido (revert vs. imagem anterior).

16.2 Recomendações de hardening (evolução do pipeline)

Área	Recomendação
Gates de segurança	Promover <code>secret_detection</code> e depois <code>sca_dependencias</code> a bloqueantes (remover <code>allow_failure</code>) — §8.4
Zero downtime	Avaliar blue-green / healthcheck + <code>--wait</code> no compose antes de derrubar o antigo
Healthcheck	Adicionar <code>healthcheck</code> no compose e endpoint Actuator restrito
Migrations	Adotar Flyway/Liquibase para versionar e aplicar schema automaticamente no start
Imagem non-root	Resolver permissão do volume <code>./arquivos</code> e reabilitar a regra <code>missing-user</code> do Semgrep
Segredos	Migrar do <code>.env</code> em disco para um cofre (Vault / GitLab masked+protected vars)
Observabilidade	Centralizar logs (Loki/ELK) e métricas (Prometheus) do container
Pin de imagens	Fixar digests de <code>trivy:latest</code> / <code>semgrep:latest</code> / <code>gitleaks:latest</code> para builds reprodutíveis

Documento gerado a partir da análise direta do repositório `api-achados-perdidos` (arquivos `.gitlab-ci.yml`, `Dockerfile`, `docker-compose.yml`, `application.properties`, `deploy/nginx/*.conf`, `scripts/*`). Mantenha este runbook versionado junto ao código e atualize-o a cada mudança no pipeline. — XPTO · Rock in Rio Backend · v1.0 · 23/07/2026.